Research and Related Work

AI-Based System-Wide Real-Time Masking of Offensive Language and People on Android Phones

Ahmed Mohammed Omer · Omer Abd Almonim · Momen Kamal — Software Engineering graduation project

18 October 2026 · Companion to research-methods.md (full procedures) · References in IEEE style

This document collects the research behind the project: the questions we ask, how we answer them, what earlier work already exists, where it falls short, and what we have found so far. Section 4 is the related-work review; it is the base of the report's background chapter.

1. Summary
2. Research questions
3. Research design and methods
4. Related work
  4.1 Offensive language detection
  4.2 Arabic offensive language
  4.3 Small text models for phones
  4.4 Text inside images
  4.5 Finding and segmenting people
  4.6 Hiding sensitive images
  4.7 The Android platform
  4.8 Existing products
  4.9 The gap we address
5. Findings so far
6. Planned evaluation
7. Threats to validity
8. Ethics, privacy and AI use
References

1 Summary

We build an Android app that covers offensive words and images of people on the screen, **across apps** (system-wide), with all AI running **on the phone**, and that covers new content **before** the user sees it. Research so far shows this is possible on a normal, not rooted, Android 14+ phone, and that no existing product or published system does all of it.

0
existing tools that filter **text** across all apps on a phone (§4.8)

163 + 364
competitor issues and reviews analysed; top complaint: content shows before the blur (§5.2)

~189 ms
one full-screen people check on a low-cost phone's GPU, sustained for 15 minutes (§5.3)

88–91%
of big people covered on test phone screens; small people only 24–54% (§5.3)

1 906
Arabic offensive terms combined from 7 open lists (§5.4)

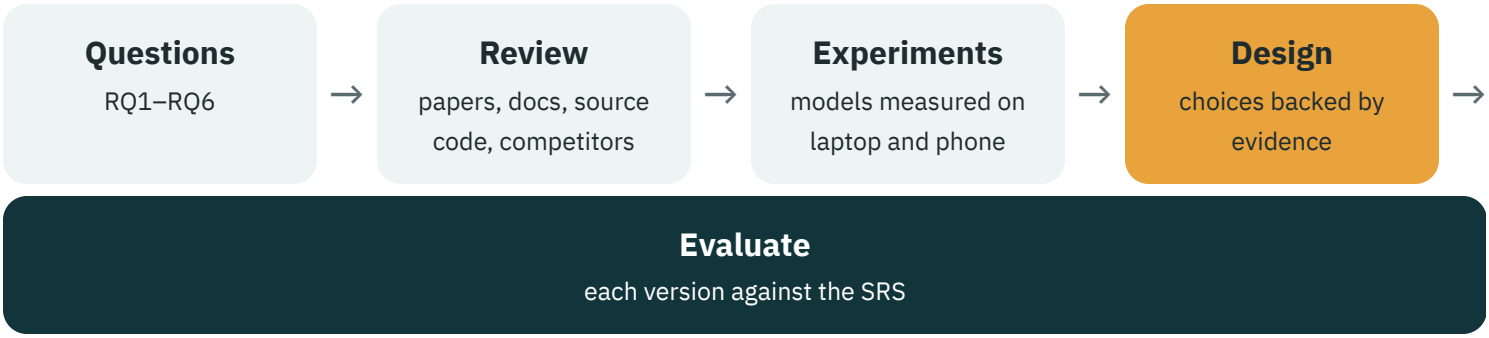
Contribution: a system-wide, on-device, Arabic-first filter that covers offensive words (in text and in images) and people, and hides new content until it is checked.

2 Research questions

ID	Question	Versions
RQ1	Can a normal (not rooted) Android app hide offensive text and images of people in every app, without first showing them to the user? What are the platform limits?	all
RQ2	How well does a word list with Arabic text normalization find offensive Arabic and English words on screen, and how many normal words does it cover by mistake?	v1
RQ3	How fast and how accurate are real-time person-segmentation models on a low-cost phone, for people of different sizes?	v3–v5
RQ4	What problems do users report with existing tools, and which can a different design avoid?	all
RQ5	Can a small phone model, trained with labels from large open models ("teachers"), find offensive messages **and the offensive words in them** better than a word list, within the phone's limits?	v1
RQ6	Can on-phone OCR read Arabic and English text inside images fast enough to cover the offensive words in it?	v2

3 Research design and methods

The project follows **engineering design research**: build the app in stages and evaluate each stage against measurable requirements (SRS v2.2). Seven methods feed it.



#	Method	Answers	Status
M1	Literature and technical documentation review	RQ1–RQ3, RQ6	Done; updated as needed
M2	Source-code analysis (Android, Chromium, a competitor)	RQ1, RQ4	Done
M3	Competitor and user-feedback analysis (thematic coding)	RQ4	Done
M4	Controlled experiments with person-segmentation models	RQ3	Done (laptop, phone accuracy, phone speed)
M5	Lexicon construction (Arabic offensive words)	RQ2	Combined list done; manual review planned
M6	Evaluation of each release on the test phone	RQ1–RQ3, RQ6	Planned (v1 → v4, v5 bonus)
M7	Training our own on-device text model (teacher → student)	RQ2, RQ5	Planned (v1)

Procedures, data sizes, software versions and limits of each method: research-methods.md §4.

4 Related work

Our project sits where eight lines of work meet. For each one we summarise what exists, what we reuse, and what is missing for our goal.

Language

- 4.1 Offensive language detection
- 4.2 Arabic offensive language
- 4.3 Small text models for phones
- 4.4 Text inside images

Vision

- 4.5 Finding and segmenting people
- 4.6 Hiding sensitive images

System and products

- 4.7 The Android platform
- 4.8 Existing products
- 4.9 The gap we address

4.1 Offensive language detection

Automatic detection of abusive and hateful language is a mature research area. Schmidt and Wiegand survey the early methods, from word lists and n-gram features to learned classifiers [72]. Davidson *et al.* show that separating hate speech from merely offensive language is hard, and that word lists alone flag many harmless posts [71]. The OffensEval shared tasks fixed common labels for offensive language and attracted hundreds of teams; the 2020 edition added Arabic, Danish, Greek and Turkish, with pre-trained transformers on top [73], [74]. Most systems decide for a **whole post**; the toxic-spans task asks instead **which words** are offensive [60], which is what a screen filter needs to cover only those words.

> **Gap.** These systems label posts offline, usually on a server, for moderators. None of them works on whatever text is on the user's screen, in real time, on the phone, and marks the exact words to cover.

4.2 Arabic offensive language

Arabic adds dialects, spelling variants, letter normalization, elongation and Arabizi (Arabic in Latin letters). Mubarak *et al.* built one of the first Arabic obscene-word lists from social media [1]. The OSACT4 shared task (2020) created the standard Arabic offensive-tweet benchmark [2]; strong systems relied on careful preprocessing [3], [4], and the winning ALT system used character n-grams with linear models [64]. OSACT5 (2022) extended it to fine-grained hate speech [75]. Bensalem *et al.* review 54 Arabic toxic-language datasets and note that most come from Twitter and a few dialects [76]; Arabizi needs its own handling [5]. Arabic pre-trained encoders such as MARBERT [61] and CAMeLBERT [62] are strong on these tasks. Recent work targets safety for Arabic LLMs and memes: ArabicDialectSafety [56], the ArGuard shared task [57] and FanarGuard [59]. A distilled Arabic toxicity model made for phones exists (HAYAA-student, message level only) [15]. Open Arabic word lists exist but are small and scattered [6], [20]–[24].

What we reuse: the OSACT data for training and testing; Lucene's Arabic normalization rules [17]; all open word lists, combined (§5.4); multi-phrase matching in one pass with Aho–Corasick [18], [94].

> **Gap.** No public Arabic resource is both **word-level** (which words to cover) and **small enough for a low-cost phone**. Context-dependent words such as كلب (dog) or كافر (unbeliever) are offensive only in some sentences, which a list cannot decide.

4.3 Small text models for phones

Knowledge distillation trains a small "student" to copy a large "teacher" [77]; DistilBERT keeps most of BERT's quality at 40% less size [78], and the HAYAA student keeps 96.7% agreement with a teacher 68× larger [15]. Large language models now label text well: ChatGPT beat crowd workers on several annotation tasks [79], and open guard models such as Qwen3Guard cover 119 languages [54]. **Decision models** such as Jev return a fixed, typed choice with a confidence instead of free text, which is fast and easy to threshold [49]; open alternatives include Laya, Von, Kev and GLiNER2.5-Decide [50]–[53]. Jev runs only as a cloud API, so we take the idea, not the model: our phone model gives a typed answer (category + confidence + which words), and large decision and guard models are used only offline, as teachers that label training data. Multilingual models spend most of their size on the word table; **vocabulary trimming** removes unused tokens with no retraining [63]. Small candidates include multilingual encoders (mmBERT [55], multilingual E5 [69], EmbeddingGemma [67]), static embeddings (Model2Vec [65], [66]) and small generative models (Gemma 3 270M [68]). **Quantization** to 8-bit integers shrinks and speeds models on phones [86], but must be checked: our own int8 people model found nobody (§5.3).

> **Gap.** Teacher → student has not been applied to **word-level Arabic offensive language** under a phone budget (≤ 100 MB RAM, ≤ 50 ms per message). That is our RQ5.

4.4 Text inside images

Memes, thumbnails and video subtitles carry text that screen readers never see. PP-OCR introduced an ultra-light OCR pipeline for mobile [80]; PaddleOCR 3.0 and PP-OCRv5 add multilingual recognition with models under 100 M parameters [81], [16], and an open-source Android app already runs it [41]. Google's on-device ML Kit OCR does **not** support Arabic [33]. In our design, text read from an image goes through the same word list and text model as screen text (§4.3). The ArGuard task studies harmful Arabic memes, but with server-side models [57].

> **Gap.** Reading Arabic text in images **on a low-cost phone, fast enough to cover words while scrolling** is not measured anywhere we found. That is our RQ6 (step P0d).

4.5 Finding and segmenting people

Real-time detection began with one-stage detectors such as YOLO [82]. Mask R-CNN added a pixel mask for each object [83], and YOLACT made instance masks real-time [84]. Today's small real-time segmenters are the YOLO11/YOLO26 family [10] and RF-DETR [8], [11]. SAM 3 segments objects from a text prompt such as "person" and can label training data [9]. Phone-oriented options exist but do not fit screens: MediaPipe's segmenters are made for portraits and video calls at 256 × 256 px input [96], so a medium person on a shrunk screen becomes a few pixels; MobileSAM [97] and EdgeSAM [98] are fast but need a point or box prompt for each object; FastSAM [99] segments every object rather than people. We therefore keep a people-trained YOLO segmenter on the phone and use SAM only offline, for labelling. COCO is the standard benchmark, with small, medium and large object sizes [7]. Small objects are a known weakness; **slicing the image into tiles** (SAHI) raises small-object accuracy by 5–7 AP points without retraining [85], which is our v4 method. AI Benchmark shows how widely phone chips differ, and that phone AI units often do not help in practice [87], which matches our test phone (§5.3).

> **Gap.** Benchmarks use photos and fast GPUs. There are no published results for **people on phone screens** (feeds, grids, thumbnails) on a **low-cost phone**, which is why we built our own tests (M4).

4.6 Hiding sensitive images

Most image filters classify the **whole image** as safe or not, for example pornography classifiers [88] and the browser model nsfwjs used by a competitor (§4.8). Two findings shape how we hide:

- **Light blur is not enough.** McPherson *et al.* recognised faces and objects through blur and pixelation with deep learning [89]. We use **solid covers**.
- **Even a glimpse counts.** People understood the meaning of pictures shown for only **13 ms** [90]. A filter that shows content and then blurs it has already failed. We **cover first, then check**.

> **Gap.** Image-level filters blur or block a whole picture after it is shown. We cover **only the person's shape**, and **before** it is shown.

4.7 The Android platform

A normal app cannot change how Android draws the screen. Three platform features make our design possible:

Feature	What it gives	Source
Accessibility service	On-screen text from every app that shares it, and an event when it changes	[25], [35]
Per-character boxes (Android 8+)	The exact screen position of each word, so only the word is covered; Chrome provides them for web pages	[26], [34]
Window picture (Android 14+)	A picture of one app's window **without our covers**, at most one every 333 ms	[27], [28]
Accessibility overlay	Trusted covers that may be fully solid while touches pass through	[27], [29]

The limits exist for security. Cloak and Dagger showed that an app with the overlay and accessibility permissions could take full control of the phone's interface [91]. Android then capped see-through overlays of normal apps at 80% opacity [29], [30], requires consent for each screen-capture session [31], and Google Play restricts non-accessibility uses of the accessibility API [32].

> **Consequence.** The accessibility path is the only non-root way to get solid, touch-through covers and a picture that excludes them. It needs Android 14+, and it is the reason our app asks for one permission only and has no internet permission.

4.8 Existing products

Several products hide sensitive images; none filters text system-wide.

Product	Across apps	Filters text	Covers only the person	Hidden before shown	On device
HaramBlur browser extension [37]	× websites	×	× whole image	optional; lifts after 7 s	✓
HaramBlur Android app [38]	popular apps	×	× whole image	× shows 1–3 s	✓
SafeGaze extension [39]	× websites	×	not checked	not checked	not checked
Apple Communication Safety [92]	Apple's apps	×	× whole image	✓	✓
Google Messages sensitive content warnings [93]	Messages only	×	× whole image	✓	✓
Our app	✓	✓ Arabic + English	✓ mask	✓ no timeout	✓ no internet

Apple and Google detect **nudity** in messages, a narrower goal than all people. HaramBlur guesses gender from faces; we cover all people, because a wrong guess means exposure. HaramBlur facts: our source-code reading (M2) and user-feedback analysis (M3).

4.9 The gap we address

What exists	What is missing	Our contribution
Offensive-language classifiers for posts, run on servers (4.1, 4.2)	Word-level, on-device, system-wide	A word list + small model that covers the exact words, on the phone (v1)
Large Arabic models and safety guards (4.2, 4.3)	A model sized for a low-cost phone	Teacher → student, measured on the test phone (RQ5)
Mobile OCR without Arabic, or with Arabic but unmeasured on phones (4.4)	Speed and accuracy on a low-cost phone	Measured OCR for text in images (v2, RQ6)
Person segmenters benchmarked on photos and fast GPUs (4.5)	Results for phone screens on weak phones	Our phone-screen tests and tile strategy (v3, v4, RQ3)
Image filters that blur after showing (4.6, 4.8)	Solid covers before showing	Cover first, check, then uncover (all versions)

5 Findings so far

5.1 Platform (RQ1)

Option	Verdict	Why
Screen recording + normal overlay	**Rejected**	Covers at most 80% solid; consent every session; records our own covers
Accessibility service only (Android 14+)	**Chosen**	Works across apps; one permission; solid touch-through covers; window picture without our covers; max 3 pictures/s
Root or custom system	**Rejected**	Only rooted phones; breaks with updates

Limits found: apps that block screenshots (banking, Netflix and other protected video apps) cannot be pictured, so only their screen text is filtered; apps that draw text themselves (many games) share no text; video while it plays is left for after graduation (v6); Xiaomi's battery saver can stop the service; installing from a file needs "restricted settings" on Android 13+.

5.2 Existing tools (RQ4)

We coded 163 GitHub issues of the HaramBlur extension and 364 recent Play Store reviews of its Android app (~696 000 installs, 4.61 stars). Counts come from keyword searches, then reading the matches.

Theme	Evidence	Our design answer
Content shows before the blur (most common)	~20 reviews ("takes 1–2 seconds"); issue #190: visible 1–3 s	Cover first, never lift without a check
Feels heavy	Reviews; extension using 2–5 GB RAM, 100% CPU	Work only when the screen changes
Misses images	Small images, drawings, anime, background images	Check the window picture; tiles for medium people
Blurs the wrong thing	~15 reviews: men, a drawing, a whole video	No gender guess; cover the person's mask
Asks for permission again	Our own use; reviews	One accessibility permission, granted once
Light blur shows details	Reviews	Solid covers
No text filtering	Neither product	v1 and v2

5.3 People models on a low-cost phone (RQ3)

Test phone: Xiaomi Redmi 12 (Helio G88, Mali-G52 GPU, Android 15). Eleven model variants were compared on the laptop; the chosen one (YOLO26n-seg, 640 px) was exported for the phone.

GPU

~205 ms

per full screen; 189 ms average over 15 minutes non-stop

Best CPU setting

277 ms

2 fast cores with fp16 math; heat slows the CPU up to ~30%

Phone AI unit (NNAPI)

no help

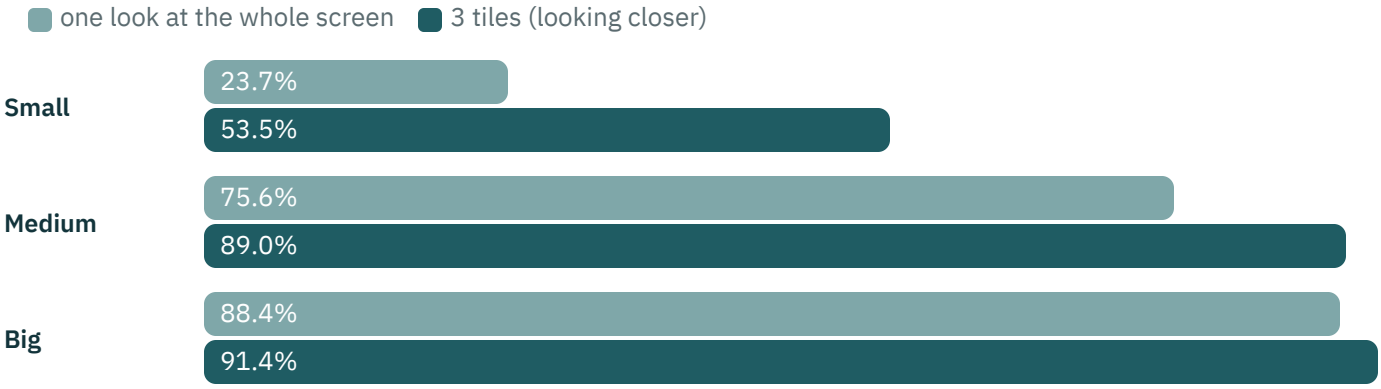
as slow as one CPU thread

Smaller input (480 px)

rejected

faster, but loses too much accuracy

People covered (at least 90% of the person under the cover), synthetic phone feed screens built from COCO photos, the phone's model file:



Lesson: the fully int8 phone model found **nobody** (0 of 448 detections), because boxes and scores shared one scale; it still looked fast. Every converted model is now checked against the original before its speed is tested.

Details: p0-results-android-v1.md, p0-results-linux-v1.md.

5.4 Arabic word list (RQ2)

1 906
unique terms from 7 open lists, normalized and de-duplicated, with the source of each term kept

947
in Arabic script (680 words, 267 phrases)

959
in Latin letters (Arabizi)

0
terms lost in the merge (checked automatically)

Next: two team members label every Arabic term independently (category, context-dependent or not), with agreement measured by Cohen's kappa [95].

6 Planned evaluation

Version	Main measures	Target
v1 text	Share of offensive words covered (OSACT test split, English comments, our screenshots); false covers on clean text (news, Quran, schoolbooks); delay before the cover (slow-motion video); battery	≥ 95% covered; ≤ 1 false cover per 1 000 words; ≤ ~3 frames
v1 text model	Macro-F1 and word-level F1 against the word list and ready-made small models; real RAM and ms per message on the phone	Better than the list; ≤ 100 MB, ≤ 50 ms
v2 text in images	OCR speed and memory on the phone; offensive words covered in real screenshots	Measured (P0d)
v3, v4 people	People covered by size on real phone screenshots; waiting time; exposure frames	≥ 95% big (v3) and medium (v4); wait ≤ 0.7 s

Results are reported with 95% confidence intervals. Full plan: research-methods.md §4.6–§4.7.

7 Threats to validity

Threat	What we do
Synthetic test screens differ from real ones	v3–v5 are evaluated on real phone screenshots
One test phone	Report the model; repeat key measurements on a second phone where possible
Heat changes speed	Pauses, temperature recorded, sustained 15-minute test
Word lists from one region's social media	Manual review; tests on standard-Arabic clean text
Teacher labels can be wrong	Several teachers, majority vote, human-checked sample with kappa
Keyword counts of reviews are approximate	All low-star reviews read in full

8 Ethics, privacy and AI use

- **Privacy by design:** screen content is processed in memory on the phone; the app has no internet permission and saves nothing.
- **Public data:** reviews and issues are reported in aggregate, without user names. A competitor's AGPL code was read for ideas, not copied.
- **Licenses** of every word list, dataset and model are recorded; one list's author will be asked for permission.
- **Private training screenshots** (v5) are never published or sent to outside services.
- **AI tools** helped with searching, scripts and drafting; all measurements come from our own runs, and the team checks and owns the content. The department allows AI tools.

References

Numbers [1]–[70] are the same as in research-methods.md §9, so citations agree across documents; the full list, including references not cited here, is kept there. NEW marks references added for this review (accessed 18 October 2026). Other online sources were accessed on 6 October 2026.

[1] H. Mubarak, K. Darwish, and W. Magdy, "Abusive language detection on Arabic social media," in *Proc. First Workshop on Abusive Language Online*, Vancouver, Canada, 2017, pp. 52–56, doi: 10.18653/v1/W17-3008.

[2] H. Mubarak, K. Darwish, W. Magdy, T. Elsayed, and H. Al-Khalifa, "Overview of OSACT4 Arabic offensive language detection shared task," in *Proc. 4th Workshop on Open-Source Arabic Corpora and Processing Tools*, Marseille, France, 2020, pp. 48–52.

[3] F. Husain, "OSACT4 shared task on offensive language detection: Intensive preprocessing-based approach," in *Proc. 4th Workshop on Open-Source Arabic Corpora and Processing Tools*, 2020, pp. 53–60.

[4] F. Husain, "Arabic offensive language detection using machine learning and ensemble machine learning approaches," arXiv:2005.08946, 2020.

[5] I. Bensalem, M. Mout, and P. Rosso, "Offensive language detection in Arabizi," in *Proc. ArabicNLP 2023*, Singapore, 2023, pp. 423–434, doi: 10.18653/v1/2023.arabicnlp-1.36.

[6] M. Attia, "arabic-offensive-words" (LLM-based Arabic offensive lexicon), GitHub, 2025. https://github.com/mohammedattia/arabic-offensive-words

[7] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, 2014. arXiv:1405.0312.

[8] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, "RF-DETR: Neural architecture search for real-time detection transformers," arXiv:2511.09554, 2025.

[9] N. Carion *et al.*, "SAM 3: Segment anything with concepts," arXiv:2511.16719, 2025.

[10] Ultralytics, "Ultralytics YOLO" (YOLO11 and YOLO26 segmentation), v8.4.171. https://github.com/ultralytics/ultralytics

[11] Roboflow, "RF-DETR," v1.11.1. https://github.com/roboflow/rf-detr

[15] MahmoudMabrok, "HAYAA Student — Arabic toxicity classifier for on-device use," Hugging Face. https://huggingface.co/MahmoudMabrok/HAYAA-student-int8

[16] PaddlePaddle, "PaddleOCR: On-device deployment" (PP-OCRv5). https://www.paddleocr.ai/v3.3.1/en/version3.x/deployment/on_device_deployment.html

[17] Apache Software Foundation, "Apache Lucene: ArabicNormalizer." https://lucene.apache.org/core/8_5_1/analyzers-common/org/apache/lucene/analysis/ar/ArabicNormalizer.html

[18] R. Bor, "aho-corasick" (Java implementation), GitHub. https://github.com/robert-bor/aho-corasick

[20] Shutterstock / LDNOOBW, "List of dirty, naughty, obscene, and otherwise bad words," CC-BY-4.0. https://github.com/LDNOOBW/List-of-Dirty-Naughty-Obscene-and-Otherwise-Bad-Words

[21] LDNOOBWV2, "List of dirty, naughty, obscene and otherwise bad words, V2," CC0. https://github.com/LDNOOBWV2/List-of-Dirty-Naughty-Obscene-and-Otherwise-Bad-Words_V2

[22] UXBERT Labs, "Arabic bad & dirty word filter list," MIT. https://github.com/uxbertlabs/arabic_bad_dirty_word_filter_list

[23] 4troDev, "profanity.csv: A multilingual profanity database," MIT. https://github.com/4troDev/profanity.csv

[24] T. Wormer, "cuss" (Arabic in Latin letters), MIT. https://github.com/words/cuss

[25] Android Developers, "AccessibilityService." https://developer.android.com/reference/android/accessibilityservice/AccessibilityService

[26] Android Developers, "AccessibilityNodeInfo" and API 26 differences. https://developer.android.com/reference/android/view/accessibility/AccessibilityNodeInfo

[27] Android Developers, "API 34 differences: AccessibilityService" (takeScreenshotOfWindow, accessibility overlays). https://developer.android.com/sdk/api_diff/34/changes/android.accessibilityservice.AccessibilityService

[28] Android Open Source Project, AccessibilityService.java and AbstractAccessibilityServiceConnection.java (android11, android13, android14, master). https://android.googlesource.com/platform/frameworks/base/

[29] Android Open Source Project, "Cap alpha to max if SAW + FLAG_NOT_TOUCHABLE," commit a451d5b. https://android.googlesource.com/platform/frameworks/base/+/a451d5bae2e2

[30] Android Developers, "Tapjacking." https://developer.android.com/topic/security/risks/tapjacking

[31] Android Developers, "Media projection." https://developer.android.com/media/grow/media-projection

[32] Google, Play Console Help: use of the AccessibilityService API, answer 10964491. https://support.google.com/googleplay/android-developer/answer/10964491

[33] Google, ML Kit text recognition v2, supported languages. https://developers.google.com/ml-kit/vision/text-recognition/v2/languages

[34] The Chromium Authors, WebContentsAccessibilityImpl.java (main). https://chromium.googlesource.com/chromium/src/

[35] Google Codelabs, "Developing an accessibility service for Android." https://codelabs.developers.google.com/codelabs/developing-android-a11y-service

[37] M. Alganzory, "HaramBlur" browser extension, source code (commit f47000e) and issue tracker. https://github.com/alganzory/HaramBlur

[38] M. Alganzory, "HaramBlur" Android app, v2.0.5, Google Play. https://play.google.com/store/apps/details?id=com.haramblur

[39] "SafeGaze" browser extension, Chrome Web Store. https://chrome.google.com/webstore/detail/safegaze-browser-extensio/aaddkkmpkpcnchdggomfaehpagbekalb

[41] Ente, "The best open-source Android OCR." https://ente.com/blog/ocr/

[49] TypeSafe AI, "Jev" decision model (Jev 1.13, released Sep. 2026). https://openrouter.ai/typesafe/jev-1.13/api

[50] DataCamp, overview of open-source alternatives to TypeSafe Jev (Laya, Von, Kev, NanoJev, and others). https://www.datacamp.com/blog/top-open-source-jev-alternatives

[51] ConvAI Innovations, "Laya" and "Laya-multilingual," Apache-2.0, Hugging Face. https://huggingface.co/convaiinnovations/laya-multilingual

[52] LiteRT Community, "Laya-Multilingual-LiteRT" (phone build with measured latency), Hugging Face. https://huggingface.co/litert-community/Laya-Multilingual-LiteRT

[53] Fastino, "GLiNER2.5-Decide," Apache-2.0, Hugging Face. https://huggingface.co/fastino/gliner2.5-decide

[54] H. Zhao *et al.*, "Qwen3Guard technical report," arXiv:2510.14276, 2025.

[55] M. Marone *et al.*, "mmBERT: A modern multilingual encoder with annealed language learning," arXiv:2509.06888, 2025.

[56] W. Zaghouani, M. R. Biswas, K. K. Aldous, and M. Bessghaier, "ArabicDialectSafety: A dialect-aware benchmark for Arabic content safety classification," arXiv:2608.01291, 2026.

[57] F. Alam *et al.*, "ArGuard shared task: Harmful content detection in Arabic memes and LLM prompts," arXiv:2609.29349, 2026.

[59] M. Fatehkia, E. Altinisik, and H. T. Sencar, "FanarGuard: A culturally-aware moderation filter for Arabic language models," arXiv:2511.18852, 2025.

[60] J. Pavlopoulos, J. Sorensen, L. Laugier, and I. Androutsopoulos, "SemEval-2021 Task 5: Toxic spans detection," in *Proc. SemEval-2021*, 2021, pp. 59–69, doi: 10.18653/v1/2021.semeval-1.6.

[61] M. Abdul-Mageed, A. Elmadany, and E. M. B. Nagoudi, "ARBERT & MARBERT: Deep bidirectional transformers for Arabic," in *Proc. ACL-IJCNLP 2021*, doi: 10.18653/v1/2021.acl-long.551.

[62] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, and N. Habash, "The interplay of variant, size, and task type in Arabic pre-trained language models," in *Proc. Sixth Arabic NLP Workshop*, 2021.

[63] A. Ushio, Y. Zhou, and J. Camacho-Collados, "Efficient multilingual language model compression through vocabulary trimming," in *Findings of EMNLP 2023*, pp. 14725–14739, doi: 10.18653/v1/2023.findings-emnlp.981.

[64] S. Hassan *et al.*, "ALT submission for OSACT shared task on offensive language detection," in *Proc. 4th Workshop on Open-Source Arabic Corpora and Processing Tools*, 2020, pp. 61–65.

[65] MinishLab, "Model2Vec: Fast state-of-the-art static embeddings." https://github.com/MinishLab/model2vec

[66] MinishLab, "potion-multilingual-128M," and NAMAA-Space, "Zarra," Hugging Face.

[67] H. Schechter Vera *et al.*, "EmbeddingGemma: Powerful and lightweight text representations," arXiv:2509.20354, 2025.

[68] Google, "Introducing Gemma 3 270M: The compact model for hyper-efficient AI," Google Developers Blog.

[69] L. Wang *et al.*, "Multilingual E5 text embeddings: A technical report," arXiv:2402.05672, 2024.

[71] T. Davidson, D. Warmsley, M. Macy, and I. Weber, "Automated hate speech detection and the problem of offensive language," in *Proc. ICWSM*, 2017, pp. 512–515. **NEW**

[72] A. Schmidt and M. Wiegand, "A survey on hate speech detection using natural language processing," in *Proc. Fifth Int. Workshop on Natural Language Processing for Social Media*, Valencia, Spain, 2017, pp. 1–10, doi: 10.18653/v1/W17-1101. **NEW**

[73] M. Zampieri, S. Malmasi, P. Nakov, S. Rosenthal, N. Farra, and R. Kumar, "SemEval-2019 Task 6: Identifying and categorizing offensive language in social media (OffensEval)," in *Proc. SemEval-2019*, 2019, pp. 75–86, doi: 10.18653/v1/S19-2010. **NEW**

[74] M. Zampieri *et al.*, "SemEval-2020 Task 12: Multilingual offensive language identification in social media (OffensEval 2020)," in *Proc. Fourteenth Workshop on Semantic Evaluation*, 2020, pp. 1425–1447, doi: 10.18653/v1/2020.semeval-1.188. **NEW**

[75] H. Mubarak, H. Al-Khalifa, and A. Al-Thubaity, "Overview of OSACT5 shared task on Arabic offensive language and hate speech detection," in *Proc. 5th Workshop on Open-Source Arabic Corpora and Processing Tools*, Marseille, France, 2022, pp. 162–166. https://aclanthology.org/2022.osact-1.20 **NEW**

[76] I. Bensalem, P. Rosso, and H. Zitouni, "Toxic language detection: A systematic review of Arabic datasets," arXiv:2312.07228, 2024. **NEW**

[77] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015. **NEW**

[78] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019. NEW

[79] F. Gilardi, M. Alizadeh, and M. Kubli, "ChatGPT outperforms crowd workers for text-annotation tasks," *Proc. Natl. Acad. Sci. USA*, vol. 120, no. 30, e2305016120, 2023, doi: 10.1073/pnas.2305016120. NEW

[80] Y. Du *et al.*, "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020. NEW

[81] C. Cui *et al.*, "PaddleOCR 3.0 technical report," arXiv:2507.05595, 2025. NEW

[82] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788. NEW

[83] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE ICCV*, 2017, doi: 10.1109/ICCV.2017.322. NEW

[84] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-time instance segmentation," in *Proc. IEEE/CVF ICCV*, 2019, pp. 9157–9166. NEW

[85] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *Proc. IEEE ICIP*, Bordeaux, France, 2022, pp. 966–970, doi: 10.1109/ICIP46576.2022.9897990. NEW

[86] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF CVPR*, 2018, pp. 2704–2713. NEW

[87] A. Ignatov *et al.*, "AI Benchmark: All about deep learning on smartphones in 2019," in *Proc. IEEE/CVF ICCV Workshops*, 2019, pp. 3617–3635, doi: 10.1109/ICCVW.2019.00447. NEW

[88] M. Moustafa, "Applying deep learning to classify pornographic images and videos," arXiv:1511.08899, 2015. NEW

[89] R. McPherson, R. Shokri, and V. Shmatikov, "Defeating image obfuscation with deep learning," arXiv:1609.00408, 2016. NEW

[90] M. C. Potter, B. Wyble, C. E. Hagmann, and E. S. McCourt, "Detecting meaning in RSVP at 13 ms per picture," *Atten. Percept. Psychophys.*, vol. 76, no. 2, pp. 270–279, 2014, doi: 10.3758/s13414-013-0605-z. NEW

[91] Y. Fratantonio, C. Qian, S. P. Chung, and W. Lee, "Cloak and dagger: From two permissions to complete control of the UI feedback loop," in *Proc. IEEE Symp. Security and Privacy*, 2017, pp. 1041–1057, doi: 10.1109/SP.2017.39. NEW

[92] Apple, "About Communication Safety on your child's Apple device," Apple Support, article 105069. https://support.apple.com/en-us/105069 NEW

[93] Google, "Sensitive content warnings in Google Messages," Google Messages Help, answer 15724426. https://support.google.com/messages/answer/15724426 NEW

[94] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Commun. ACM*, vol. 18, no. 6, pp. 333–340, 1975, doi: 10.1145/360825.360855. NEW

[95] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104. NEW

[96] Google, "MediaPipe Image Segmenter: models," Google AI Edge. https://developers.google.com/edge/mediapipe/solutions/vision/image_segmenter NEW

[97] C. Zhang *et al.*, "Faster Segment Anything: Towards lightweight SAM for mobile applications," arXiv:2306.14289, 2023. **NEW**

[98] C. Zhou, X. Li, C. C. Loy, and B. Dai, "EdgeSAM: Prompt-in-the-loop distillation for on-device deployment of SAM," arXiv:2312.06660, 2023. **NEW**

[99] X. Zhao *et al.*, "Fast Segment Anything," arXiv:2306.12156, 2023. **NEW**